

User Guide for OpenFOAM-8

*Volume-averaged porous-media modeling of catalytic packed-bed
reactors with detailed surface chemistry: Implementation and
validation of porousPBRFoam*

Please cite our article if you are using this package: J. H. Lee, D. N. Nguyen, H. W. Seo, H. J. Ahn, C. S. Yoo, Volume-averaged porous-media modeling of catalytic packed-bed reactors with detailed surface chemistry: Implementation and validation of porousPBRFoam (2026)(submitted).

Contents

1	General descriptions	3
1.1	Library for surface chemistry in OpenFOAM	3
1.2	porousPBRFoam solver	4
1.3	surfChemkinToFoam utility	6
2	Running simulations using porousPBRFoam solver	7
2.1	Case file description for non-reacting flow simulation	7
2.1.1	Settings at time directory	7
2.1.2	Settings at constant directory	9
2.1.3	Settings at system directory	12
2.2	Case file description for reacting flow simulation	12
2.2.1	Settings at time directory	14
2.2.2	Settings at constant directory	15
2.2.3	Settings at system directory	18
2.3	Preparation of thermodynamic and reaction input files	18
2.3.1	Gas-phase input files	18
2.3.2	Site-phase input files	19
2.4	Running simulation using porousPBRFoam	22
3	Extension of porous-media closure models	23
3.1	Pressure drop closure models	24
3.2	Thermal-conductivity closure models	27
3.3	Species diffusivity closure models	30
3.4	Wall heat-transfer closure models	33

1. General descriptions

The framework is fully native OpenFOAM(OF) and comprises newly developed libraries and solvers as follows:

- libraries:
 - **thermophysicalModels** includes surface chemistry sub-library.
- solvers:
 - **porousPBRFoam**
- utilities:
 - **surfChemkinToFoam**

1.1. Library for surface chemistry in OpenFOAM

Figure 1 presents the class diagram of the updated **thermophysicalModels** library, including the **surfaceChemistryModels** sub-library used for heterogeneous surface chemistry. The present implementation is built upon the surface-chemistry framework developed in the previous **surfaceChemistryFoam** library [1] and extends it to support porous-media modeling (PMM) in **porousPBRFoam**. In the diagram, green boxes with circled numbers indicate representative classes selected through the OpenFOAM run-time selection mechanism. Yellow boxes denote the classes developed in the previous **surfaceChemistryFoam** framework, whereas red boxes indicate the newly developed classes introduced in the present work for PMM.

The yellow classes provide the general infrastructure for heterogeneous surface chemistry, including surface-reaction classes, surface reaction-rate models, catalyst thermophysical models, surface-chemistry ODE solvers, and catalytic-boundary conditions. Since these components were developed in the previous **surfaceChemistryFoam** framework [1], detailed descriptions of the yellow classes are not repeated here. Readers are referred to the *surfaceChemistryFoam_userGuide_OF8.pdf* provided in the directory **porousPBRFoam-8/docum**
entations [2] for the detailed class descriptions and implementation details.

The following description therefore focuses on the red classes newly introduced for the PMM-based surface-chemistry formulation in `porousPBRFoam`.

- *PorousMediaSurfaceChemistryModel* is the main class for the PMM surface-chemistry formulation in `porousPBRFoam`. It evaluates heterogeneous surface reactions locally in each porous cell. The species production rates and heat-release rate per unit catalytic surface area obtained from the surface chemistry are converted into volumetric source terms through surface-to-volume scaling using the specified surface-to-volume ratio.
- The *porousMediaHeatTransferWall* class provides the wall boundary condition for PMM-based heat-transfer simulations in `porousPBRFoam`. It evaluates heat exchange between the reactor wall and the porous bed based on the wall heat-transfer coefficient and the effective thermal conductivity of the porous medium.

1.2. *porousPBRFoam* solver

The `porousPBRFoam` solver is developed for the simulation of reacting flows in PBRs using a porous-media modeling (PMM) approach. In this approach, the packed bed is represented as an effective continuum, and the macroscopic governing equations are obtained by applying volume averaging to the microscopic fluid- and solid-phase balance equations. This averaging introduces unclosed terms arising from spatial fluctuations of the microscopic fields and fluid–solid interfacial transport. These terms are described using porous-media closure models. The solver accounts for gas flow, pressure drop, heat transfer, species transport, and heterogeneous surface chemistry within the porous region. The volumetric species-production and heat-release rates evaluated by *PorousMediaSurfaceChemistryModel* are incorporated as source terms into the macroscopic species and energy equations, respectively. The governing equations implemented in `porousPBRFoam` are provided in the manuscript.

The principal case-specific settings for the PMM formulation are specified in the `porousMediaProperties` and `surfaceChemistryProperties` dictionaries. The `porousMediaProperties` dictionary specifies the physical properties of the packed bed, including

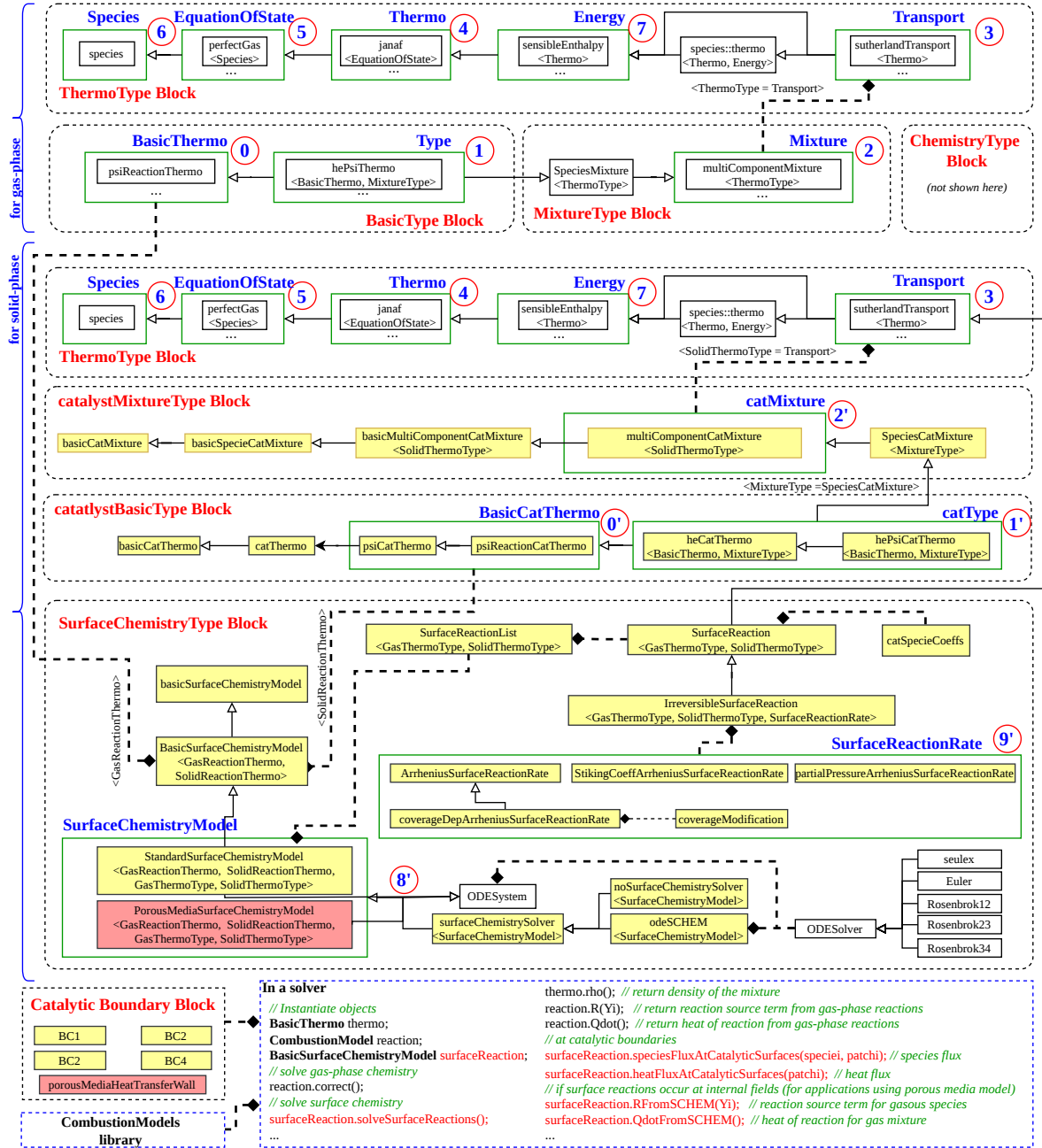


Figure 1: Class diagram of the extended thermophysical library used by `porousPBRFoam`. Yellow boxes denote classes originally developed in `surfaceChemistryFoam` for detailed heterogeneous surface chemistry, whereas red boxes denote classes developed in the present work for PMM. Boxes with circled numbers indicate representative classes selected through the run-time selection mechanism. Solid arrows indicate inheritance relationships, whereas dashed arrows indicate usage relationships between classes.

the porosity, particle diameter, and solid-phase thermophysical properties. It also specifies the closure models and their required parameters for pressure drop, effective thermal conductivity, effective species diffusivity, and wall heat transfer. In contrast, the `surfaceChemistryProperties` dictionary specifies the settings for the heterogeneous surface-chemistry calculation, including the surface-chemistry model, its numerical solution parameters, and the surface-to-volume ratio used to convert the surface-based reaction rates into volumetric source terms.

For the gas phase, `porousPBRFoam` follows the standard OpenFOAM thermophysical framework used in `reactingFoam`. In the present implementation, the gas-phase thermophysical properties are evaluated using the Sutherland transport model, JANAF thermodynamics, and the perfect-gas equation of state. A specific combination of thermodynamics, transport, equation-of-state, energy, and mixture models is supported, as summarized in Table 1.

Table 1: An available combination of gas-phase thermophysical models supported by `porousPBRFoam`

Option	type	mixture	transport	thermo	energy	equationOfState	specie
a	hePsiThermo	multi ^{a2}	STM ^{a3}	janaf	senE/senI ⁵	perfectGas	specie

where all abbreviations are:

senE/senI⁵: `sensibleEnthalpy/sensibleInternalEnergy`;

multi^{a2}: `multiComponentMixture`; STM^{a3}: `sutherland`;

1.3. *surfChemkinToFoam utility*

Based on the original OpenFOAM `chemkinToFoam` utility, a new preprocessing utility named `surfChemkinToFoam` has been developed for heterogeneous surface chemistry. This utility automatically converts surface-chemistry input files from CHEMKIN format into OpenFOAM format. Detailed instructions for using this utility are provided in Section 2.3.2.

2. Running simulations using porousPBRFoam solver

2.1. Case file description for non-reacting flow simulation

The first tutorial case, `NonReactingPBRHeatTransfer`, demonstrates a non-reacting heat-transfer simulation in a packed-bed reactor (PBR) using `porousPBRFoam`. The case is available in `tutorials/NonReactingPBRHeatTransfer` within the `porousPBRFoam-8` source repository [2]. It introduces the basic porous-media modeling (PMM) setup, including the specification of packed-bed parameters, heat-transfer closure models, and the wall heat-transfer boundary condition for the temperature field.



Figure 2: Case file structure for the non-reacting heat-transfer simulation using `porousPBRFoam`.

As shown in Fig. 2, the case-directory structure follows the standard `reactingFoam` layout. The primary addition for the PMM formulation is the `porousMediaProperties` dictionary, which specifies the packed-bed properties and the porous-media closure models.

2.1.1. Settings at time directory

For the non-reacting heat-transfer case, the fields in the initial-time directory can be configured in the same manner as those in a standard `reactingFoam` case. The primary

additional consideration concerns the boundary condition for the temperature field, T , when heat exchange between the reactor wall and the porous bed is modeled using the effective thermal conductivity of the porous medium.

Setting for the temperature field. For the temperature field T , standard boundary conditions such as `fixedValue` and `zeroGradient` can be used in the same manner as in `reactingFoam`. When wall-to-bed heat transfer is modeled using the porous-media formulation, however, the reactor-wall boundary condition should be specified as `porousMediaHeatTransferWall`. This boundary condition evaluates the heat exchange between the reactor wall and the porous bed using the specified wall heat-transfer closure and the effective thermal conductivity of the porous medium, k_{eff} , rather than the molecular thermal conductivity of the gas phase.

```

/*-----* C++ *-----*\
=====
\\      / F ield           | OpenFOAM: The Open Source CFD Toolbox
\\    /  O peration       | Website:  https://openfoam.org
\\  /    A nd              | Version:   8
\\//     M anipulation    |
/*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       T;
}
// * * * * *

dimensions      [0 0 0 1 0 0 0];

internalField    uniform 293;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 293;
    }
    wall
    {
        type      porousMediaHeatTransferWall;
        mode      coefficient;
        kappaMethod fluidThermo;
        h          uniform 100;
        Ta         constant 373;
        value      $internalField;
    }
}

```



```

    }
    ...
}
// *****

```

2.1.2. Settings at constant directory

The `porousMediaProperties` dictionary is required in the `constant` directory for all PMM simulations using `porousPBRFoam`, regardless of whether heterogeneous surface reactions are considered. It is the only additional dictionary required for the non-reacting heat-transfer case relative to a standard `reactingFoam` case. This dictionary specifies the solver mode, solid-phase thermophysical properties, packed-bed geometrical parameters, superficial velocity, and the closure models for effective thermal conductivity, wall heat transfer, and pressure drop.

An example `porousMediaProperties` dictionary for the Wen heat-transfer case is shown below.

```

/*-----* C++ -*-----*\
=====
\\      / F i e l d      | OpenFOAM: The Open Source CFD Toolbox
\\    /  O p e r a t i o n | Website:  https://openfoam.org
\\  /    A n d             | Version:   8
\\//     M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       porousMediaProperties;
}
// *****

solverMode                PM;

// Solid Thermophysical properties
SolidThermalConductivity  1.05;    // [W/m-K]
SolidDensity              2500;    // [kg/m3]
SolidHeatCapacity         840;     // [J/kg-K]

// Porous Media geometric properties
Porosity                  0.4165; // [-]
ParticleDiameter          5e-3;   // [m]
ReactorDiameter           4.1e-2; // [m]
SuperficialVelocity       1.235;  // [m/s]

// Porous Media heat transfer model
pressureDropClosureModel  Ergun;

```

```

ThermalConductivityClosureModel      Wakao ;
StagnantThermalConductivityClosureModel  ZS ;
WallHeatTransferClosureModel          Li ;
DiffusivityClosureModel               BruggemanWilhelm ;

// *****

```

The principal entries in the `porousMediaProperties` dictionary are described below.

- `solverMode` selects the operating mode of `porousPBRFoam`. The PM option is used for non-reacting PMM simulations, whereas `PMSCHEM` is used for reacting PMM simulations involving heterogeneous surface chemistry.

The solid-phase thermophysical properties and the packed-bed geometrical and flow parameters are specified as follows:

- `SolidThermalConductivity` (k_s) specifies the thermal conductivity of the solid phase.
- `SolidDensity` (ρ_s) specifies the density of the solid phase.
- `SolidHeatCapacity` ($c_{p,s}$) specifies the specific heat capacity of the solid phase.
- `Porosity` (ε) specifies the void fraction of the packed bed.
- `ParticleDiameter` (d_p) specifies the representative particle diameter used in the porous-media closure models.
- `ReactorDiameter` (d_{PBR}) specifies the reactor diameter required by applicable wall heat-transfer closure models.
- `SuperficialVelocity` ($|\langle \mathbf{u} \rangle|$) specifies the magnitude of the superficial velocity through the packed bed. It is used to evaluate velocity-dependent porous-media closure models.

The closure-model selectors determine how the porous-media closure terms are evaluated. The formulations of the available models, including those used in the validation cases presented in this work, are summarized in the manuscript. Users may also modify the existing correlations or implement additional closure models for other particle geometries, packing structures, and operating conditions, as described in Section 3.

- **pressureDropClosureModel** selects the closure model used to evaluate the momentum resistance and resulting pressure drop across the packed bed. The available options are *Darcy*, *Forchheimer*, and *Ergun*. The *Darcy* model uses a prescribed viscous resistance coefficient, whereas the *Forchheimer* model uses prescribed viscous and inertial resistance coefficients. The *Ergun* model evaluates the viscous and inertial resistance terms from the packed-bed porosity and particle diameter using the Ergun correlation.
- **DiffusivityClosureModel** selects the closure model used to evaluate the effective species diffusivity, $D_{i,\text{eff}}$, in the packed bed. The currently available option is *BruggemanWilhelm*, which combines the Bruggeman correction for molecular diffusion through the porous structure with the Wilhelm correlation for flow-induced mechanical dispersion.
- **ThermalConductivityClosureModel** selects the closure model used to evaluate the effective thermal conductivity of the packed bed, k_{eff} . The available options are *Const*, *Standard*, *Bunnell*, *Yagi*, and *Wakao*. The *Const* model uses the prescribed value of **EffectiveThermalConductivity**, whereas the *Standard* model evaluates k_{eff} by weighting the fluid- and solid-phase thermal conductivities according to their respective volume fractions. The *Bunnell*, *Yagi*, and *Wakao* models employ packed-bed correlations that account for flow-induced thermal dispersion. The *Yagi* model additionally requires the model parameters **YagiAlphaBeta** and **YagiPhi**. The *Wakao* model requires a stagnant-bed thermal-conductivity closure, which is selected using **StagnantThermalConductivityClosureModel**.
- **StagnantThermalConductivityClosureModel** selects the closure model used to evaluate the stagnant effective thermal conductivity, k_{eff}^0 , required by the *Wakao* model. The available options are *Kunii*, *Krupiczka*, and *ZS*. These models evaluate k_{eff}^0 using the packed-bed properties and the fluid- and solid-phase thermal conductivities according to their respective correlations.
- **WallHeatTransferClosureModel** selects the closure model used to evaluate the wall

heat-transfer coefficient, h_w , governing heat exchange between the packed bed and the reactor wall. The available options are *Const*, *Yagi*, *Li*, and *Bey*. The *Const* model uses the prescribed value of `heatTransferCoefficient`, whereas the *Yagi*, *Li*, and *Bey* models evaluate h_w using empirical packed-bed wall heat-transfer correlations.

2.1.3. Settings at system directory

The files located in the `system` directory contain the numerical and simulation-control settings required for running the case. In general, their configuration is identical to that used in `reactingFoam`, and no special modifications are typically required for catalytic reacting-flow simulations.

2.2. Case file description for reacting flow simulation

The second tutorial case, `IsoThermalNH3Decomposition`, demonstrates a reacting-flow simulation of NH_3 decomposition in a packed-bed reactor (PBR) using `porousPBRFoam`. The case is available in `tutorials/IsoThermalNH3Decomposition` within the `porousPBRFoam-8` source repository [2]. It corresponds to the isothermal NH_3 decomposition validation case presented in the manuscript and extends the porous-media modeling (PMM) setup introduced in the non-reacting heat-transfer tutorial by activating heterogeneous surface chemistry. Accordingly, additional fields and dictionaries for the surface-chemistry calculation are required in addition to the `porousMediaProperties` dictionary used for the PMM formulation.

As shown in Fig. 3, the case-directory structure follows the standard `reactingFoam` layout but includes additional fields and dictionaries required for heterogeneous surface chemistry. The `porousMediaProperties` dictionary is also required to specify the packed-bed properties and porous-media closure models, as described for the non-reacting tutorial case.

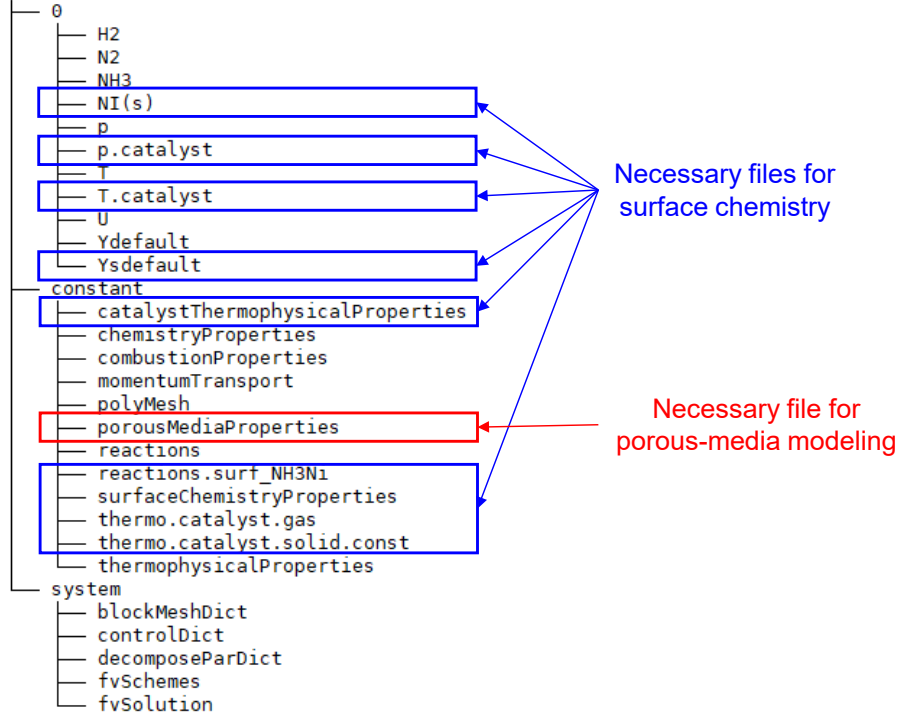


Figure 3: Case file structure for reacting-flow simulations using `porousPBRFoam`. Additional files required for surface chemistry and porous-media modeling are highlighted separately.

In the initial-time directory (e.g., `0`), additional fields are introduced for the surface-chemistry calculation, while the gas-phase fields are configured in the same manner as those in a standard `reactingFoam` case. The files `p.catalyst` and `T.catalyst` define the pressure and temperature fields used for the catalyst phase, respectively. The `Ysdefault` file provides the initial site-species fraction field, whereas individual site-species fields, such as `NI(s)` and `Ru(s)`, specify the initial site fractions of the corresponding surface species. The required site-species fields depend on the heterogeneous reaction mechanism employed in the simulation. By default, the intermediate site-species fractions are initialized to zero, although nonzero initial values may be specified when required.

In the `constant` directory, the additional input files required for the surface-chemistry calculation are `catalystThermophysicalProperties`, `surfaceChemistryProperties`, `reactions.surf`, `thermo.catalyst.solid`, and `thermo.catalyst.gas`. These files

specify the catalyst thermophysical properties, the surface-chemistry calculation settings, the heterogeneous reaction mechanism, and the thermodynamic data used by the surface-chemistry solver. The `porousMediaProperties` dictionary is separately required for the PMM formulation. Its keywords and available closure-model options are not repeated here because they were described in the non-reacting tutorial setup.

2.2.1. Settings at time directory

Setting for site fraction fields of surface species. The site-fraction fields must be initialized such that their sum is nonzero. Typically, the vacant-site species, such as `NI(s)`, is assigned an initial site fraction of 1.0, while the adsorbed surface species are initialized to zero. Alternatively, nonzero initial fractions may be assigned to multiple site species when required. If all site-fraction fields are initialized to zero, the surface-chemistry model may terminate during initialization. Standard boundary conditions, such as `zeroGradient`, may be applied at the domain boundaries.

```

/*-----* C++ -*-----*\
=====
\\      / F i e l d      |   OpenFOAM: The Open Source CFD Toolbox
\\      / O p e r a t i o n   |   Website:  https://openfoam.org
\\      / A n d               |   Version:   8
\\      / M a n i p u l a t i o n |
/*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       NI(s);
}
// *****
dimensions      [0 0 0 0 0 0 0];

internalField   uniform 1;

boundaryField
{
    heatedWall
    {
        type          zeroGradient;
    }
    inlet
    {
        type          zeroGradient;
    }
    ...
}

```

```

}
// *****

```

Settings for the temperature fields (T and $T.catalyst$). The gas-phase temperature field T is configured using the porous-media heat-transfer formulation described in Section 2.1.1. The auxiliary field $T.catalyst$, which is required by the surface-chemistry library, can be created by copying T and assigning the `zeroGradient` condition to its boundary patches. The values of $T.catalyst$ are not used to evaluate the heterogeneous reaction rates in the PMM formulation.

Settings for the pressure fields (p and $p.catalyst$). The gas-phase pressure field p is configured in the same manner as in the base flow solver. The auxiliary field $p.catalyst$, which is required by the surface-chemistry library, can be created by copying p and assigning appropriate boundary conditions, such as `zeroGradient`. The values of $p.catalyst$ are not used to evaluate the heterogeneous reaction rates in the PMM formulation.

2.2.2. Settings at constant directory

Setting for thermophysicalProperties file. The `thermophysicalProperties` dictionary specifies the thermodynamic, transport, equation-of-state, and energy models used for the gas phase in `porousPBRFoam`. Its configuration generally follows that of a standard `reactingFoam` case. As illustrated in Fig. 4(b), the currently supported gas-phase configuration uses *heP-siThermo* with *multiComponentMixture*, *sutherland* transport, *janaf* thermodynamics, and the *perfectGas* equation of state, together with either *sensibleEnthalpy* or *sensibleInternalEnergy* as the energy variable. These model names must be specified exactly as shown because their combination determines the thermophysical-model type constructed by the solver.

a) <i>catalystThermophysicalProperties</i> file (for surface-phase)			b) <i>thermophysicalProperties</i> file (for gas-phase)		
<code>solidThermoType</code>	keyword	Selected model	<code>thermoType</code>	keyword	Selected model
{			{		
1' <code>solidType</code>		hePsiCatThermo;	1 <code>type</code>		hePsiThermo;
2' <code>solidMixture</code>		multiComponentCatMixture;	2 <code>mixture</code>		multiComponentMixture;
3' <code>solidTransport</code>		const;	3 <code>transport</code>		sutherland;
4' <code>solidThermo</code>		hConst;	4 <code>thermo</code>		janaf;
5' <code>solidEnergy</code>		sensibleEnthalpy;	5 <code>energy</code>		sensibleEnthalpy;
6' <code>solidEquationOfState</code>		rhoConst;	6 <code>equationOfState</code>		perfectGas;
7' <code>solidSpecie</code>		specie;	7 <code>specie</code>		specie;
}			}		
<code>inertSpecie</code> AR;			<code>inertSpecie</code> AR;		
<code>#include "thermo.catalyst.solid"</code>			<code>#include "thermo.catalyst.gas"</code>		

Figure 4: An example settings of the `catalystThermophysicalProperties` file (a) and the `thermophysicalProperties` file (b) for simulations using the `porousPBRFoam` solver.

Setting for catalystThermophysicalProperties file. The `catalystThermophysicalProperties` dictionary specifies the thermophysical-model configuration required for the catalyst phase and surface species. The currently supported configuration is illustrated in Fig. 4(a). The model names must be specified exactly as shown to construct the catalyst thermophysical model supported by `porousPBRFoam`.

Setting for chemistryProperties file. The `chemistryProperties` dictionary is configured identically to that used in a standard `reactingFoam` case.

Setting for surfaceChemistryProperties file. The `surfaceChemistryProperties` dictionary specifies the heterogeneous surface-chemistry model and its numerical integration settings. In `porousPBRFoam`, the heterogeneous reaction rates are evaluated within the porous region and converted from rates per unit catalytic surface area into volumetric source terms using the specified surface-to-volume ratio.

```

/*-----* C++ *-----*/
=====
\\      /  F i e l d
\\      /  O p e r a t i o n
\\      /  A n d
\\      /  M a n i p u l a t i o n
\\      /
/*-----*/

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
}

```



```

    object      surfaceChemistryProperties;
}
// * * * * *

// Select the porous-media surface-chemistry treatment
onlySurfaceCatalyst false; // default: false

// Compatibility entries; leave empty for the present PMM setup
catalystRegions
{
    catalyticSurfaces    ( );
    catalyticZones       ( );
}

// Catalytic surface area per unit reactor volume [1/m]
SurfaceToVolumeRatio    2.8E5;

// Surface-chemistry method and ODE solver
surfaceChemistryType
{
    method      porousMediaSCHEM;    // porous-media model
    solver      odeSCHEM;
}

chemistry      on;

initialChemicalTimeStep 1e-07;

odeSCHEMCoeffs
{
    solver      seulex;
    absTol      1e-8;
    relTol      0.01;
}

#include "reactions.surf"
// *****

```

For the PMM formulation, `onlySurfaceCatalyst` is set to `false`. The `catalystRegions` sub-dictionary is retained for compatibility with the surface-chemistry library; its `catalyticSurfaces` and `catalyticZones` entries are not used in the present PMM setup and are therefore left empty.

The `SurfaceToVolumeRatio` keyword specifies the catalytic fluid–solid interfacial area per unit porous-region volume. This quantity converts the species-production and heat-release rates per unit catalytic surface area into the corresponding volumetric source terms used in the porous-media governing equations. For a packed bed, its value should be specified consistently with the catalyst loading, particle size, porosity, and catalytically active surface area represented in the simulation.

The `surfaceChemistryType` sub-dictionary selects the surface-chemistry method and solver. The `method` entry must be set to *porousMediaSCHEM* for the PMM formulation. The `solver` entry selects the surface-chemistry solver, for which only `odeSCHEM` is currently supported. When `odeSCHEM` is selected, the `odeSCHEMCoeffs` sub-dictionary specifies the ODE integrator and its numerical tolerances. In the example above, `seulex` is selected with absolute and relative tolerances of 10^{-8} and 10^{-2} , respectively.

The `chemistry` keyword activates or deactivates the surface-chemistry calculation, while `initialChemicalTimeStep` specifies the initial time step used for its ODE integration. The surface-reaction mechanism is included using the `#include "reactions.surf"` directive. The procedure for preparing this mechanism file in OpenFOAM format is described in Section 2.3.2.

Finally, the catalytic surface reaction mechanism in OpenFOAM format is included using the `#include "reactions.surf"` directive. The procedure for preparing this mechanism file is described in Section 2.3.2.

2.2.3. Settings at system directory

The files located in the `system` directory contain the numerical and simulation-control settings required for running the case. In general, their configuration is identical to that used in `reactingFoam`, and no special modifications are typically required for catalytic reacting-flow simulations.

2.3. Preparation of thermodynamic and reaction input files

2.3.1. Gas-phase input files

The gas-phase input files are prepared following the standard OpenFOAM procedure used for `reactingFoam`. The gas-phase reaction mechanism is provided through the `reactions` file included in `chemistryProperties`, together with the corresponding thermodynamic data file. The species names must be consistent between the reaction mechanism and thermodynamic data. No additional preprocessing is required for `porousPBRFoam`.

2.3.2. Site-phase input files

Two input files are required for the surface phase: a surface-reaction mechanism file in OpenFOAM format, such as `reactions.surf`, and a thermodynamic-data file for the site species, such as `thermo.catalyst.solid`. The `reactions.surf` file defines the heterogeneous surface reactions and their kinetic parameters. An example is shown below:

```
reactions
{
    un-named-reaction-1
    {
        type            irreversibleSurfaceArrheniusSurface;
        reaction         "H(s) + H(s) = NI(s) + NI(s) + H2";
        A                3.315E+18;
        beta             0;
        Ta               9887.53396;
    }
}
```

The `thermo.catalyst.solid` file specifies the thermodynamic and transport properties of the site species. An example format is given below:

```
species
9
(
    NI(s)
    NH3(s)
    H(s)
    N(s)
    OH(s)
    O(s)
    NH2(s)
    NH(s)
    H2O(s)
);

NI(s)
{
    specie
    {
        molWeight      58.7;
        siteDensity     2.6e-8;
    }
    equationOfState
    {
        rho            8.9e3;
    }
    thermodynamics
    {
        Cp              440;
        Hf               17.48e6;
        Cv              440;
    }
    transport
```

```

    {
        mu          0;
        Pr          1;
    }
}
...

```

The `reactions.surf` and `thermo.catalyst.solid` files can be generated using the `surfChemkinToFoam` utility:

```
$ surfChemkinToFoam <input1> <input2> <input3> <output1> <output2>
```

where:

- `input1`: surface-chemistry mechanism file in CHEMKIN format, e.g., `surf.inp`
- `input2`: thermodynamic-data file for surface (site) species in CHEMKIN format, e.g., `surfTherm.dat`
- `input3`: OpenFOAM dictionary file containing the thermodynamic and transport-property definitions for the `const` thermodynamic combination of surface species, e.g., `surfaceThermoDict`
- `output1`: surface-chemistry mechanism file converted into OpenFOAM format, with a user-defined file name such as `reactions.surf`
- `output2`: thermodynamic- and transport-property file for the surface species, with a user-defined file name such as `thermo.catalyst.solid`.

The first two input files in CHEMKIN format can be obtained directly from the desired surface-chemistry mechanism available in the literature. In particular, the surface-reaction mechanism file (`surf.inp`) supports several kinetic-constant model keywords implemented in the present framework. The currently supported kinetic-constant models include:

- Arrhenius kinetic constant model (AKC model)
- Coverage-dependent kinetic constant model (CKC model)

- Sticking-coefficient kinetic constant model (SKC model)
- Partial-pressure dependent kinetic constant model (PKC model)

Example definitions for each corresponding kinetic-constant model in the CHEMKIN mechanism file are provided below.

AKC model

```
2H(s) => 2Rh(s) + H2          5.574E+19  0.239  59.691
```

CKC model

```
C0(s) => Rh(s) + C0          1.300E+13  0.295  134.070
COV/C0(s)          0.000E+00  0.000  -47.000/
```

SKC model

```
2Rh(s) + H2 => 2H(s)          3.000E-02  0.000  0.000
STICK
```

PKC model

```
2Ru(s) + H2 => 2H(s)          2.689E+12  0.000  0.0000
PKC
```

The third input file, `surfaceThermoDict`, specifies the thermodynamic and transport properties assigned to the site species for the `const` thermodynamic combination during the conversion process. The required thermodynamic and transport properties of the site species, such as `rho`, `Cp`, and `Cv`, are directly read from the `surfaceThermoDict` file. An example `surfaceThermoDict` file is shown below.

```
surfaceThermoDefaults
{
    // equationOfState
    rho      8900;          // kg/m3

    // transport
    mu       0;             // kg/m/s
    Pr       1;

    // thermodynamics, used only when surfaceThermoModel == const
    Cp       440;           // J/kg/K
    Cv       440;           // J/kg/K
    Hf       0;             // J/kg
}
```

The generated catalyst thermodynamic data are written to `thermo.catalyst.solid`. This file is included in the `catalystThermophysicalProperties` dictionary, as shown in

Figure 4. The generated surface reaction mechanism is written to `reactions.surf`. This file is included in the `surfaceChemistryProperties` dictionary, as described in Section 2.2.2.

At `porousPBRFoam-8/tutorials/surfChemkinToFoam/example/` [2] directory, an example is provided to demonstrate the use of the `surfChemkinToFoam` utility. It is important to note that using `surfChemkinToFoam` requires only three input files placed in the case directory. In other words, `0`, `constant`, and `system` directories are not needed in this step.

To generate the `reactions.surf` and `thermo.catalyst.solid` files, first navigate to the example directory:

```
$ cd ~/OpenFOAM/yourDirectory/porousPBRFoam-8/tutorials/surfChemkinToFoam/
↪ example/
```

Then, execute the following command in the terminal. The command-line arguments must be specified in the correct order:

```
$ surfChemkinToFoam surf.inp surfTherm.dat surfaceThermoDict reactions.surf
↪ thermo.catalyst.solid
```

Several examples for converting commonly used surface chemical mechanisms are also available in `porousPBRFoam-8/tutorials/surfChemkinToFoam/` directory [2]. In each directory, a script `Allrun` is provided to automate converting files for convenient.

Since the conversion process can sometimes be sensitive to the formatting and consistency of the CHEMKIN input files, an additional document entitled *ManuallyPrepareSurfaceReactionsFile.pdf* is provided in the directory `porousPBRFoam-8/documentations` [2]. This document is intended to help users debug the conversion process or manually prepare the surface-reaction mechanism file in OpenFOAM format, particularly when the reaction mechanism size is relatively small.

2.4. Running simulation using porousPBRFoam

Once boundary conditions are correctly specified, all required files are in place and the case is ready to run. To start the simulation, first generate the computational mesh using an appropriate utility (e.g., `blockMesh`). After the mesh has been created, run the solver from the case directory in the same manner as `reactingFoam` by typing:

`$ porousPBRFoam`

Two tutorial cases are provided in the `porousPBRFoam-8/tutorials/` directory [2]. The `NonReactingPBRHeatTransfer` tutorial demonstrates the basic PMM setup for non-reacting heat transfer, including the configuration of the `porousMediaProperties` dictionary and the wall heat-transfer boundary condition. The `IsoThermalNH3Decomposition` tutorial demonstrates the reacting-flow PMM setup incorporating heterogeneous surface chemistry. Users are encouraged to examine these tutorials to become familiar with the case configuration and execution procedure before applying `porousPBRFoam` to other PBR simulations.

3. Extension of porous-media closure models

As noted in the preceding sections, the porous-media closure models implemented in `porousPBRFoam` can be modified or extended to represent different particle geometries, packing structures, and operating conditions. This section describes the source-code structure used to implement the closure models and provides examples of how users can modify the existing models or add new ones.

The primary source files discussed in this section are located within the `porousPBRFoam-8/` directory as follows:

- `applications/solvers/porousPBRFoam/createFieldsPorousMedia.H`
- `applications/solvers/porousPBRFoam/updateForPorousMedia.H`
- `src/catalyticSurfaceBoundaries/porousMediaHeatTransferWall/porousMediaHeatTransferWallFvPatchScalarField.C`

The paths above are specified relative to the root directory of the `porousPBRFoam-8` repository. Prescribed porous-medium parameters and other quantities that remain constant throughout a simulation are declared, validated, and initialized in `createFieldsPorousMedia.H`. Thermophysical properties and closure quantities that depend on the local solution fields are recalculated during the simulation in `updateForPorousMedia.H`. These quantities

include the porous momentum-source term, $\mathbf{S}_m$, the effective thermal conductivity, k_{eff} , and the effective species diffusivity, $D_{i,\text{eff}}$.

The wall heat-transfer coefficient, h_w , is evaluated separately in `porousMediaHeatTransferWallFvPatchScalarField.C`, which implements the `porousMediaHeatTransferWall` boundary condition.

3.1. Pressure drop closure models

The *Darcy* pressure-drop closure is used to demonstrate the basic procedure for implementing and extending a closure model. Its relatively simple formulation illustrates how prescribed model parameters are initialized and how closure coefficients depending on the local solution fields are updated. More elaborate pressure-drop closures can be implemented following the same procedure.

For the *Darcy* closure, the porous momentum-source term is expressed as

$$\mathbf{S}_m = -\mu \mathcal{D} \langle \mathbf{u} \rangle,$$

where $\mathcal{D}$ is the Darcy resistance coefficient. This coefficient is prescribed and remains constant throughout the simulation. It is therefore declared, validated, and initialized in `createFieldsPorousMedia.H`.

Before the model-specific parameters are initialized, the presence and validity of the `pressureDropClosureModel` entry are checked as follows:

```
// Read and validate the pressure-drop closure.
if (!porousMediaDict.found("pressureDropClosureModel"))
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Missing required entry "
        << "'pressureDropClosureModel'." << nl
        << "Valid options are: None, Darcy, "
        << "Forchheimer, Ergun"
        << exit(FatalIOError);
}

word deltaPClosureModel_
(
    porousMediaDict.lookup("pressureDropClosureModel")
);

if
(
    deltaPClosureModel_ != "None"
```



```

&& deltaPClosureModel_ != "Darcy"
&& deltaPClosureModel_ != "Forchheimer"
&& deltaPClosureModel_ != "Ergun"
)
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown pressureDropClosureModel: "
        << deltaPClosureModel_ << nl
        << "Valid options are: None, Darcy, "
        << "Forchheimer, Ergun"
        << exit(FatalIOError);
}

```

The parameters required by the selected closure are then read and initialized in the corresponding model branch. For the *Darcy* closure, this procedure is implemented as follows:

```

else if (deltaPClosureModel_ == "Darcy")
{
    if (!porousMediaDict.find("DarcyCoeff"))
    {
        FatalIOErrorInFunction(porousMediaDict)
            << "pressureDropClosureModel = Darcy "
            << "requires entry 'DarcyCoeff'."
            << exit(FatalIOError);
    }

    DarcyCoeff = dimensionedScalar
    (
        "DarcyCoeff",
        dimensionSet(0, -2, 0, 0, 0, 0, 0),
        readScalar(porousMediaDict.lookup("DarcyCoeff"))
    );

    ForchCoeff = dimensionedScalar
    (
        "ForchCoeff",
        dimensionSet(0, -1, 0, 0, 0, 0, 0),
        0.0
    );

    Info<< "DarcyCoeff = "
        << DarcyCoeff.value() << " [1/m2]" << endl;
}

```

Although $\mathcal{D}$ remains constant, the coefficient $\mu\mathcal{D}$ multiplying the superficial velocity varies with the local dynamic viscosity. It is therefore recalculated during the simulation in `updateForPorousMedia.H`:

```

else if (deltaPClosureModel_ == "Darcy")
{
    tmp<volScalarField> tmu = thermo.mu();
    const volScalarField& mu = tmu();

    porousMomentumCoeff = mu*DarcyCoeff;
}

```

A new pressure-drop closure can be introduced by extending both the initialization and update procedures. Suppose that `NewModelForDeltaP` is used as a placeholder for the new closure name. First, the model name must be added to the validity check in `createFields` `PorousMedia.H`:

```
if
(
    deltaPClosureModel_ != "None"
    && deltaPClosureModel_ != "Darcy"
    && deltaPClosureModel_ != "Forchheimer"
    && deltaPClosureModel_ != "Ergun"
    && deltaPClosureModel_ != "NewModelForDeltaP"
)
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown pressureDropClosureModel: "
        << deltaPClosureModel_ << nl
        << "Valid options are: None, Darcy, "
        << "Forchheimer, Ergun, NewModelForDeltaP"
        << exit(FatalIOError);
}
```

A corresponding initialization branch must then be added in the same file:

```
else if (deltaPClosureModel_ == "NewModelForDeltaP")
{
    // Check and read the prescribed model parameters
    // from porousMediaDict.

    // Validate the parameter values.

    // Evaluate resistance coefficients that remain
    // constant during the simulation.
}
```

here, `NewModelForDeltaP` must be replaced by the name of the new closure. The initialization branch should check the presence and validity of all required dictionary entries and evaluate quantities that depend only on the prescribed porous-medium properties. Any additional variables required by the new closure must also be declared before the model-specific initialization branches.

The new model must also be included in the model-selection structure in `updateForPorousMedia.H`, where `porousMomentumCoeff` is evaluated using the required local thermophysical and flow fields. For example, if the new closure can be expressed using the same Darcy and Forchheimer coefficient form as the existing *Forchheimer* and *Ergun* closures, it can be added to their update branch:

```

else if
(
    deltaPClosureModel_ == "Forchheimer"
|| deltaPClosureModel_ == "Ergun"
|| deltaPClosureModel_ == "NewModelForDeltaP"
)
{
    tmp<volScalarField> tmu = thermo.mu();
    const volScalarField& mu = tmu();

    porousMomentumCoeff =
        mu*DarcyCoeff
        + 0.5*rho*mag(U)*ForchCoeff;
}

```

If the new closure has a different mathematical form, a separate update branch should instead be introduced to evaluate `porousMomentumCoeff`. After implementing the new closure, `porousPBRFoam` must be recompiled to incorporate the modified source code into the solver executable. From the solver directory, run:

```

$ cd ~/OpenFOAM/yourDirectory/porousPBRFoam-8/applications/solvers/porousP
↪ BRFoam
$ wmake

```

The recompiled executable can then be used with the new closure name specified for `pressureDropClosureModel` in `porousMediaProperties`.

3.2. Thermal-conductivity closure models

The *Standard* effective-thermal-conductivity closure is used to demonstrate the basic procedure for implementing and extending a closure model. Its relatively simple formulation illustrates how prescribed porous-medium properties and local thermophysical fields are combined to evaluate the effective thermal conductivity. More elaborate thermal-conductivity closures can be implemented following the same procedure. For the *Standard* closure, the effective thermal conductivity is expressed as

$$k_{\text{eff}} = \varepsilon k_f + (1 - \varepsilon)k_s, \quad (1)$$

where ε is the porosity and k_f and k_s are the thermal conductivities of the fluid and solid phases, respectively.

During field initialization, the presence and validity of the `ThermalConductivityClosureModel` entry are checked in `createFieldsPorousMedia.H` as follows:

```
// Read and validate the effective-thermal-conductivity closure.
if (!porousMediaDict.found("ThermalConductivityClosureModel"))
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Missing required entry "
        << "'ThermalConductivityClosureModel'." << nl
        << "Valid options are: Const, Standard, "
        << "Bunnell, Yagi, Wakao"
        << exit(FatalIOError);
}

word kEffClosureModel_
(
    porousMediaDict.lookup("ThermalConductivityClosureModel")
);

if
(
    kEffClosureModel_ != "Const"
    && kEffClosureModel_ != "Standard"
    && kEffClosureModel_ != "Bunnell"
    && kEffClosureModel_ != "Yagi"
    && kEffClosureModel_ != "Wakao"
)
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown ThermalConductivityClosureModel: "
        << kEffClosureModel_ << nl
        << "Valid options are: Const, Standard, "
        << "Bunnell, Yagi, Wakao"
        << exit(FatalIOError);
}
```

The porosity and solid-phase thermal conductivity are prescribed in `porousMediaProperties` and read during field initialization:

```
// Porosity [-]
scalar por
(
    readScalar(porousMediaDict.lookup("Porosity"))
);

// Solid thermal conductivity [W m^-1 K^-1]
scalar kappas
(
    readScalar
    (
        porousMediaDict.lookup("SolidThermalConductivity")
    )
);
```

These prescribed values are subsequently assigned to the corresponding computational fields, `POR` and `kappaS`. Because the *Standard* closure requires no additional model-specific param-

eters, no separate initialization branch is required for this model.

The effective thermal conductivity, `kappaEff`, and the corresponding coefficient used in the enthalpy-diffusion term, `pAlphaEff`, are updated during the simulation in `updateForPorousMedia.H` as follows:

```
else if (kEffClosureModel_ == "Standard")
{
    kappaEff = por*kappa + (1-por)*kappaS;
    pAlphaEff = kappaEff/Cp;
}
```

A new effective thermal-conductivity closure can be introduced by extending both the initialization and update procedures. Suppose that `NewModelForKeff` is used as a placeholder for the new closure name. First, the model name must be added to the validity check in `createFieldsPorousMedia.H`:

```
if
(
    kEffClosureModel_ != "Const"
    && kEffClosureModel_ != "Standard"
    && kEffClosureModel_ != "Bunnell"
    && kEffClosureModel_ != "Yagi"
    && kEffClosureModel_ != "Wakao"
    && kEffClosureModel_ != "NewModelForKeff"
)
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown ThermalConductivityClosureModel: "
        << kEffClosureModel_ << nl
        << "Valid options are: Const, Standard, "
        << "Bunnell, Yagi, Wakao, NewModelForKeff"
        << exit(FatalIOError);
}
```

If the new closure requires prescribed parameters, they should be declared, checked, and read in `createFieldsPorousMedia.H`. For example, a dimensionless empirical coefficient can be initialized as follows:

```
scalar newModelCoeff = 0.0;

if (kEffClosureModel_ == "NewModelForKeff")
{
    if (!porousMediaDict.find("NewModelCoeff"))
    {
        FatalIOErrorInFunction(porousMediaDict)
            << "ThermalConductivityClosureModel = "
            << "NewModelForKeff requires entry "
            << "'NewModelCoeff'."
            << exit(FatalIOError);
    }
}
```

```

        newModelCoeff =
            readScalar
            (
                porousMediaDict.lookup("NewModelCoeff")
            );
    }

```

The corresponding update branch must then be added in `updateForPorousMedia.H`:

```

else if (kEffClosureModel_ == "NewModelForKeff")
{
    // Evaluate the effective thermal conductivity using
    // the prescribed parameters and required local fields.
    kappaEff =
        /* expression for the new closure */;

    // Evaluate the corresponding coefficient used in
    // the enthalpy-diffusion term.
    pAlphaEff = kappaEff/Cp;
}

```

here, `NewModelForKeff` must be replaced by the name of the new closure. The new expression for `kappaEff` may use prescribed porous-medium parameters and the required local fields. The corresponding enthalpy-diffusion coefficient must then be updated as `pAlphaEff = kappaEff/Cp`.

After implementing the new closure, `porousPBRFoam` must be recompiled to incorporate the modified source code into the solver executable. From the root directory of the `porousPBRFoam-8` repository, run:

```

$ cd ~/OpenFOAM/yourDirectory/porousPBRFoam-8/applications/solvers/porousP
↪ BRFoam
$ wmake

```

The recompiled solver can then be run with the new closure name assigned to the `ThermalConductivityClosureModel` entry in `porousMediaProperties`.

3.3. Species diffusivity closure models

The *Bruggeman–Wilhelm* closure model combines molecular diffusion through the tortuous pore pathways, represented using the Bruggeman tortuosity relation, with flow-induced mechanical dispersion to evaluate the effective diffusivity of species i , $D_{i,\text{eff}}$:

$$D_{i,\text{eff}} = \sqrt{\varepsilon} D_{i,\text{mix}} + \frac{|\langle \mathbf{u} \rangle| d_p}{12\varepsilon},$$

30

where $D_{i,\text{mix}}$ is the mixture-averaged molecular diffusivity of species i , $\langle \mathbf{u} \rangle$ is the superficial velocity, and d_p is the particle diameter.

During field initialization, the presence and validity of the DiffusivityClosureModel entry are checked in createFieldsPorousMedia.H:

```
if (!porousMediaDict.found("DiffusivityClosureModel"))
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Missing required entry "
        << "'DiffusivityClosureModel'." << nl
        << "Valid option is: BruggemanWilhelm"
        << exit(FatalIOError);
}

word DiEffClosureModel_
(
    porousMediaDict.lookup("DiffusivityClosureModel")
);

if (DiEffClosureModel_ != "BruggemanWilhelm")
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown DiffusivityClosureModel: "
        << DiEffClosureModel_ << nl
        << "Valid option is: BruggemanWilhelm"
        << exit(FatalIOError);
}
```

After the closure model has been validated, a separate effective-diffusivity field is initialized for each gas-phase species:

```
PtrList<volScalarField> DiEff(Y.size());

forAll(DiEff, i)
{
    DiEff.set
    (
        i,
        new volScalarField
        (
            IOobject
            (
                "DiEff_" + composition.species()[i],
                runTime.timeName(),
                mesh,
                IOobject::NO_READ,
                IOobject::AUTO_WRITE
            ),
            mesh,
            dimensionedScalar("DiEff" + composition.species()[i],
                dimensionSet(0, 2, -1, 0, 0), 0.)
        )
    );
}
```

The effective-diffusivity fields are updated during the simulation in `updateForPorousMedia`

.H:

```
if (DiEffClosureModel_ == "BruggemanWilhelm")
{
    forAll(DiEff, i)
    {
        DiEff[i] =
            sqrt(POR + SMALL)
            *(thermophysicalTransport->alphaEff()/rho)
            + (mag(U)*DP)/(12.0*(POR + SMALL));
    }
}
```

A new species diffusivity closure can be implemented by extending the model-name validation and adding the corresponding update procedure. Here, `NewModelForDiEff` is used as a placeholder for the new closure. The model name is first added to the validity check in `createFieldsPorousMedia.H`:

```
if
(
    DiEffClosureModel_ != "BruggemanWilhelm"
    && DiEffClosureModel_ != "NewModelForDiEff"
)
{
    FatalIOErrorInFunction(porousMediaDict)
        << "Unknown DiffusivityClosureModel: "
        << DiEffClosureModel_ << nl
        << "Valid options are: BruggemanWilhelm, "
        << "NewModelForDiEff"
        << exit(FatalIOError);
}
```

If the new closure requires additional model parameters, they should be specified in `porousMediaProperties` and declared, validated, and read through `porousMediaDict` in `createFieldsPorousMedia.H`. The corresponding expression for `DiEff` must then be added to the update procedure in `updateForPorousMedia.H`:

```
else if (DiEffClosureModel_ == "NewModelForDiEff")
{
    forAll(DiEff, i)
    {
        // Obtain the molecular diffusivity or other
        // species-dependent properties required for species i.

        // Evaluate the effective diffusivity of species i.
        DiEff[i] =
            /* expression for the new closure */;
    }
}
```


here, `NewModelForDiEff` must be replaced by the name of the new closure. The update procedure must evaluate each entry of `DiEff` using the molecular diffusivity and any porous-medium or flow quantities required by the new closure.

After implementing the new closure, `porousPBRFoam` must be recompiled. From the root directory of the `porousPBRFoam-8` repository, run:

```
$ cd ~/OpenFOAM/yourDirectory/porousPBRFoam-8/applications/solvers/porousPBRFoam
↪
$ wmake
```

The recompiled solver can then be run with the new closure name assigned to the `DiffusivityClosureModel` entry in `porousMediaProperties`.

3.4. Wall heat-transfer closure models

The *Li* closure is used here to demonstrate the procedure for implementing and extending a wall heat-transfer closure model. These closures are evaluated in the `porousMediaHeatTransferWall` temperature boundary condition when it operates in `coefficient` mode. In this mode, the wall heat-transfer coefficient, h_w , is evaluated using the closure specified by the `WallHeatTransferClosureModel` entry in `porousMediaProperties`. The resulting coefficient and the prescribed temperature, T_a , are then used to impose the boundary heat-transfer condition.

For the *Li* closure, the wall heat-transfer coefficient is expressed as

$$h_w = \frac{k_f}{d_p} (0.17 Re_p^{0.79}),$$

where k_f is the fluid thermal conductivity and d_p is the particle diameter.

The wall heat-transfer closure specified by the `WallHeatTransferClosureModel` entry is read and validated in `porousMediaHeatTransferWallFvPatchScalarField.C`:

```
word hwClosureModel_
(
    porousMediaDict.lookup
    (
        "WallHeatTransferClosureModel"
```

```

    )
);

if
(
    hwClosureModel_ != "Const"
    && hwClosureModel_ != "Yagi"
    && hwClosureModel_ != "Li"
    && hwClosureModel_ != "Bey"
)
{
    FatalErrorInFunction
    (
        << "Unknown WallHeatTransferClosureModel \""
        << hwClosureModel_
        << "\" specified in porousMediaProperties."
        << nl
        << "Valid options are:" << nl
        << "    Const" << nl
        << "    Yagi" << nl
        << "    Li" << nl
        << "    Bey" << nl
        << nl
        << "Please check the dictionary entry:" << nl
        << "    WallHeatTransferClosureModel "
        << "<modelName>:" << nl
        << exit(FatalError);
    }
}

```

After validation, a Boolean flag is used to identify whether the *Li* closure has been selected:

```
bool useLiHW = (hwClosureModel_ == "Li");
```

For the *Li* closure, the superficial velocity and particle diameter are read from the **SuperficialVelocity** and **ParticleDiameter** entries in **porousMediaProperties**, respectively:

```

if (useLiHW)
{
    usup = readScalar
    (
        porousMediaDict.lookup("SuperficialVelocity")
    );

    dp = readScalar
    (
        porousMediaDict.lookup("ParticleDiameter")
    );
}

```

The fluid thermal conductivity, density, and dynamic viscosity are evaluated locally at each boundary face. The wall heat-transfer coefficient is then calculated face by face:

```

else if (useLiHW)
{
    forAll(h_, faceI)
    {
        scalar Ref =

```

```

        rhow[faceI]*dp*usup/muw[faceI];

    h_[faceI] =
        (kappaw[faceI]/dp)
        *0.17*std::pow(Ref, 0.79);
}
}

```

A new wall heat-transfer closure can be implemented by following the same structure as the *Li* closure. Suppose that `NewModelForHw` is used as a placeholder for the new closure name. First, the model name must be added to the validity check for `WallHeatTransferClosureModel`:

```

if
(
    hwClosureModel_ != "Const"
    && hwClosureModel_ != "Yagi"
    && hwClosureModel_ != "Li"
    && hwClosureModel_ != "Bey"
    && hwClosureModel_ != "NewModelForHw"
)
{
    FatalErrorInFunction
    (
        << "Unknown WallHeatTransferClosureModel \""
        << hwClosureModel_
        << "\" specified in porousMediaProperties."
        << nl
        << "Valid options are:" << nl
        << "  Const" << nl
        << "  Yagi" << nl
        << "  Li" << nl
        << "  Bey" << nl
        << "  NewModelForHw" << nl
        << exit(FatalError);
    }
}

```

A Boolean flag is then defined to identify whether the new closure has been selected:

```

bool useNewModelForHw =
    (hwClosureModel_ == "NewModelForHw");

```

If the new closure requires additional model parameters, they should be specified in `porousMediaProperties` and initialized in `porousMediaHeatTransferWallFvPatchScalarField.C`. For example, a model-specific coefficient named `NewHwCoeff` can be initialized as follows:

```

scalar newHwCoeff = 0.0;

if (useNewModelForHw)
{
    newHwCoeff = readScalar
    (
        porousMediaDict.lookup

```

```

        (
            "NewHwCoeff"
        )
    );
}

```

Finally, a model-specific branch must be added to evaluate the wall heat-transfer coefficient at each boundary face:

```

else if (useNewModelForHw)
{
    forAll(h_, faceI)
    {
        h_[faceI] =
            /* expression for the new closure */;
    }
}

```

here, `NewModelForHw` must be replaced by the name of the new closure. Any local thermo-physical properties required by the new correlation must be evaluated at the boundary faces before `h_` is updated.

After completing these modifications, the libraries containing the `porousMediaHeatTransferWall` boundary condition must be recompiled:

```

$ cd $LIB_PBR8_SRC
$ ./Allwmake

```

The recompiled boundary-condition libraries can then be used with the new closure name assigned to the `WallHeatTransferClosureModel` entry in `porousMediaProperties`.

References

- [1] D. N. Nguyen, J. H. Lee, H. W. Seo, H. J. Ahn, C. S. Yoo, `surfaceChemistryFoam`: An OpenFOAM-based library for detailed surface chemistry in reacting flow simulations, *Comput. Phys. Commun.* **327** (2026) 110290.
- [2] <https://github.com/danhnam11/porousPBRFoam-8>.